FraudLens

Explainable, calibrated credit-card fraud detection: algorithms, comparison and results

Viva report generated from the project's actual training run.

Item	Value
Model version	20261026-184402-d37e9778
Final model	LightGBM tuned (Optuna)
Dataset	Kaggle kartik2112/fraud-detection (Sparkov simulated card transactions)
Test PR-AUC (95% CI)	0.9765 (0.9719 - 0.9811)
Recall / precision at the chosen threshold	95.9% / 83.5%
Money saved vs no model (test period)	1,103,188 of 1,133,325
Code	github.com/gauravv-jainn/frauddet, folder fraudlens/
Report date	2026-10-27

Every number in this document is read from artifacts/ of the run above by scripts/build_report.py; nothing is typed by hand.

1. Summary in one page

Problem. Card fraud is rare (0.39% of test transactions) but expensive. The bank must decide, for every transaction, whether to approve it or send it to an analyst. Reviews cost money; missed fraud costs more.

Approach.

Kaggle CSVs → leakage-free features (21) → time split fit / validation / test → 4 models compared → Optuna tuning (30 trials) → isotonic calibration → cost-optimal threshold → test once + bootstrap CI → SHAP reasons → policy rules → app / API → drift monitoring

0.9765	95.9%	83.5%	7.3	97.3%
test PR-AUC	frauds caught	alerts that are fraud	false alarms / 10k genuine	of fraud money saved

Key decisions you should be able to defend:

- Time-based split; the test file (later in time) is used once.
- PR-AUC as the comparison metric because of extreme class imbalance.
- LightGBM chosen after comparing Logistic Regression, Random Forest, default LightGBM and tuned LightGBM.
- Calibrated probabilities plus a threshold that minimises money lost, not one that maximises accuracy.
- Every decision is explained with SHAP-based reasons; policy rules cover what the model cannot know.

2. Dataset

Item	Value
Source	Kaggle kartik2112/fraud-detection (Sparkov simulated card transactions)
Training file	1,296,675 transactions, 7,506 frauds (0.579%), 2019-01-01 to 2020-06-21
Test file	555,719 transactions, 2,145 frauds (0.386%), 2020-06-21 to 2020-12-31
Cards	983 in training, 924 in test
Fit / validation split	first 85% of the training period by time = fit (1,102,173 rows); last 15% = validation (194,502 rows, 1,133 frauds, from 2020-04-03)
Raw columns used	time, card number, merchant, category, amount, gender, city, state, home lat/long, city population, job, date of birth, merchant lat/long, label

The data is generated by the Sparkov simulator. It is realistic in structure but its fraud patterns are cleaner than real life; see Limitations.

3. Features (21) and how leakage is prevented

Card-behaviour features are computed after sorting by card and time, using only **strictly earlier** transactions of the same card. This is what would be known at authorisation time. A unit test (`tests/test_features.py`) recomputes features after deleting every later row and checks they are identical.

feature	group	meaning / how computed	mean |SHAP|
amt	raw	Transaction amount	0.957
log_amt	raw	log(1 + amount), a smoother version of the amount	0.203
hour	time	Hour of the day (0-23)	0.449
day_of_week	time	Day of week (0 = Monday)	0.191
is_night	time	1 if between 22:00 and 03:59	0.668
dist_home_km	geo	Haversine distance from cardholder home to merchant	0.100
age	profile	Cardholder age at transaction time	0.444
log_city_pop	profile	log population of the cardholder's city	0.486
merchant_te	encoded	Merchant target encoding: smoothed fraud rate of the merchant in the training period	0.180
n_txn_1h	velocity	Card's transactions in the previous 1 hour	0.028
n_txn_24h	velocity	Card's transactions in the previous 24 hours	0.168
amt_sum_24h	velocity	Card's total spend in the previous 24 hours	0.367
amt_to_card_mean	behaviour	Amount / card's average of all earlier amounts	0.115
amt_to_cat_mean	behaviour	Amount / card's average earlier amount in the same category	0.369
hours_since_prev	behaviour	Hours since the card's previous transaction	0.451
geo_speed_kmh	geo	km between this and the previous merchant / hours between them	0.150
impossible_travel	geo	1 if that speed > 800 km/h and the jump > 50 km	0.000
first_time_category	behaviour	1 if the card never bought in this category before	0.000
n_txn_30d	velocity	Card's transactions in the previous 30 days (replaced the lifetime counter)	0.473
n_txn_7d	spending window	Card's transactions in the previous 7 days (RFM window, Fraud Detection Handbook)	0.155
avg_amt_1d	spending window	Card's average amount over the previous day	0.524
avg_amt_7d	spending window	Card's average amount over the previous 7 days	0.223
avg_amt_30d	spending window	Card's average amount over the previous 30 days	0.167
amt_to_avg_30d	spending window	Amount / card's 30-day average ('a €5,000 purchase is suspicious if the average is €35')	0.093
amt_log_z	behaviour	z-score of log(1 + amount) against the card's own history	0.150
small_txn_24h	card testing	Card's charges under €5 in the previous 24 hours (card-testing probes)	0.256
category	categorical	Merchant category (14 values), native categorical in LightGBM	0.890
gender	categorical	Cardholder gender (as in the dataset)	0.060

4. Algorithms: what is used where, and why

algorithm / method	role	where in the code	why
Logistic Regression	Baseline classifier	fraudkit/train.py (baseline)	Simplest strong baseline; shows how much non-linearity adds
Random Forest (200 trees)	Baseline classifier	fraudkit/train.py (baseline)	Bagged trees; handles interactions, robust
LightGBM (gradient boosting)	Final model	fraudkit/train.py, fraudkit/score.py	Best accuracy on tabular data, fast, native categoricals
Optuna TPE	Hyper-parameter search	fraudkit/train.py (tuning)	Bayesian search: tries promising settings first
Stratified 3-fold CV	Tuning evaluation	fraudkit/train.py (tuning)	Every fold keeps the same 0.6% fraud rate
Smoothed target encoding (out-of-fold)	Merchant feature	fraudkit/features.py	Turns 693 merchant names into one number without leaking labels
Beta / Platt / isotonic calibration (chosen out of sample)	Probability calibration	fraudkit/score.py (Calibrator)	Makes 30% mean ~30%; needed for money decisions
Cost-minimising threshold	Decision rule	fraudkit/costs.py	Chooses the cut-off that minimises money lost
Bootstrap (1,000 resamples)	Confidence interval	fraudkit/train.py	Uncertainty of the test PR-AUC
SHAP TreeExplainer	Explanations	fraudkit/explain.py	Exact per-feature contributions for tree models
Population Stability Index	Drift monitoring	fraudkit/monitoring.py	Detects when live data stops looking like training data
Policy rules	Guardrail	fraudkit/rules.py	Covers cases outside the training data (model blind spots)

4.1 Logistic Regression (baseline)

Linear model of the log-odds of fraud. Pipeline: log(1+x) on the numeric features, median imputation with missing-value indicators, standardisation, one-hot encoding of category and gender, `LogisticRegression(C=1.0, max_iter=2000)`. It sets the floor: anything more complex must beat it.

4.2 Random Forest (baseline)

200 decision trees, each trained on a 30% bootstrap sample with a random subset of features at every split (`max_features='sqrt'`, `min_samples_leaf=5`). Predictions are averaged. It captures interactions and is robust, but it is slower and less accurate than boosting here.

4.3 LightGBM (final model)

Gradient boosting: trees are added one at a time, each fitted to the gradient of the log-loss of the current ensemble, and added with a small learning rate. LightGBM grows trees leaf-wise, uses histogram binning (fast on 1.1M rows) and handles categorical features natively. Two versions were compared: library defaults, and tuned with Optuna. The final model is trained on the fit period with all frauds and 25% of genuine rows; calibration later fixes the probability scale.

4.4 Optuna (TPE sampler)

Bayesian hyper-parameter search. Each trial trains LightGBM with 3-fold stratified cross-validation on the fit period and returns the mean PR-AUC. TPE models good and bad regions of the search space and samples where good results are likely.

hyper-parameter	search range	meaning	best value
n_estimators	150 - 1200 (step 50)	number of trees	900
learning_rate	0.01 - 0.2 (log)	shrinkage per tree	0.0550
num_leaves	15 - 255 (log)	tree complexity	23
min_child_samples	10 - 300 (log)	minimum rows per leaf (regularisation)	111
subsample	0.5 - 1.0	row sampling per tree (bagging)	0.822
colsample_bytree	0.4 - 1.0	feature sampling per tree	0.816
reg_lambda	0.001 - 10 (log)	L2 penalty on leaf weights	0.0521
reg_alpha	0.001 - 10 (log)	L1 penalty on leaf weights	0.0129
scale_pos_weight	1 - 10 (log)	extra weight on fraud rows	4.781

4.5 Calibration: three methods compared out of sample

Three calibrators are fitted on one time-half of the validation period and scored on the other (both ways): **isotonic regression** (a monotone step function), **Platt scaling** (logistic regression on the logit of the score) and **beta calibration** (logistic regression on log(p) and -log(1-p), Kull et al. 2017). The one with the lowest out-of-sample log-loss is used. Log-loss punishes confident mistakes, so a calibrator that outputs exactly 0% or 100% is penalised.

method	log_loss	brier	distinct_values	share_at_floor	chosen
isotonic	0.00250	0.000382	22	97.7%	
platt	0.00159	0.000374	190,927	0.0%	yes
beta	0.00162	0.000374	193,417	0.0%	

Why this matters: the first version used isotonic regression. Being a step function, it gave about 95% of test transactions exactly 0% and the riskiest exactly 100%, with only 94 distinct values for 555,719 transactions, so scores looked binary and the expected-loss policy (probability x amount) collapsed to 0 for most transactions. A smooth calibrator keeps the model's full ranking: 522,809 distinct probabilities on the test set now.

4.6 Cost-optimal threshold

For every possible threshold on validation: cost = missed fraud amount + 5 x flagged count. The minimum is at **0.5000**. Risk bands: Medium starts at the threshold, High at 0.3, Critical at 0.7.

4.7 SHAP TreeExplainer

Exact Shapley values for tree ensembles. For each transaction: base value + sum of feature contributions = model log-odds (verified by a test). Contributions are grouped and turned into sentences with the real numbers.

4.8 Population Stability Index (drift)

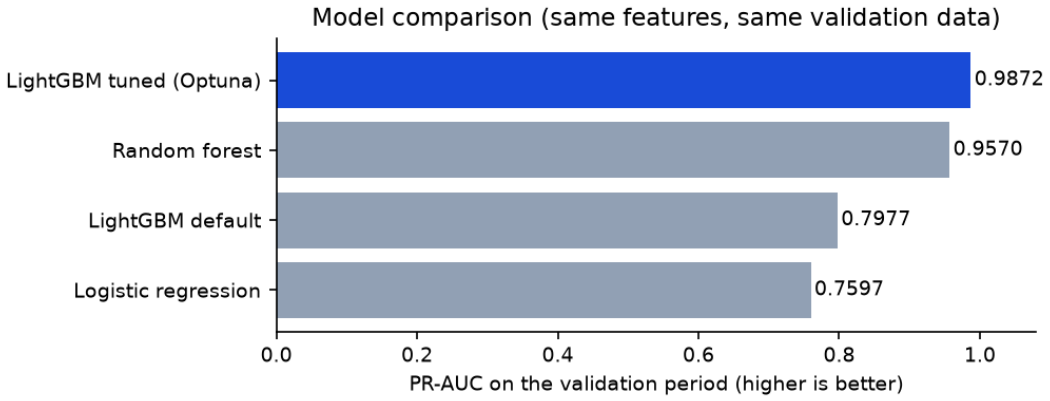
PSI = Σ (actual% - expected%) · ln(actual% / expected%) over training-time decile bins. Below 0.10 stable, 0.10-0.25 moderate, 0.25 or more significant.

5. How the comparison is done

> **Protocol.** All four models get the same 21 features and are trained on the same fit period (1,102,173 rows). They are compared on the same validation period (194,502 rows, later in time) with the same metric (PR-AUC). The winner is chosen on validation only. Calibration and threshold are also chosen on validation. Only then is the test file scored, once.

Why PR-AUC. With 0.39% fraud, accuracy is meaningless: "always genuine" is 99.61% accurate. ROC-AUC is also flattering (all models score above 0.94) because its false-positive rate is divided by hundreds of thousands of genuine transactions. PR-AUC (average precision) measures how clean the alert list is at every recall level; a random model scores the fraud rate (0.386%).

Also reported: ROC-AUC (ranking quality), Brier score (probability accuracy; lower is better), fit time.

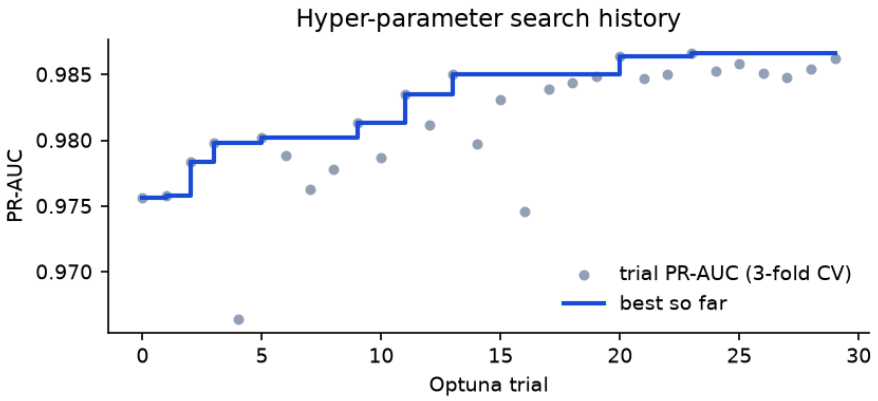


model	pr_auc	roc_auc	brier	fit_seconds	selected
LightGBM tuned (Optuna)	0.9872	0.9998	0.00053	5.4	selected
Random forest	0.9570	0.9962	0.00098	35.9	
LightGBM default	0.7977	0.9580	0.00173	2.5	
Logistic regression	0.7597	0.9814	0.00231	6.8	

Reading the comparison

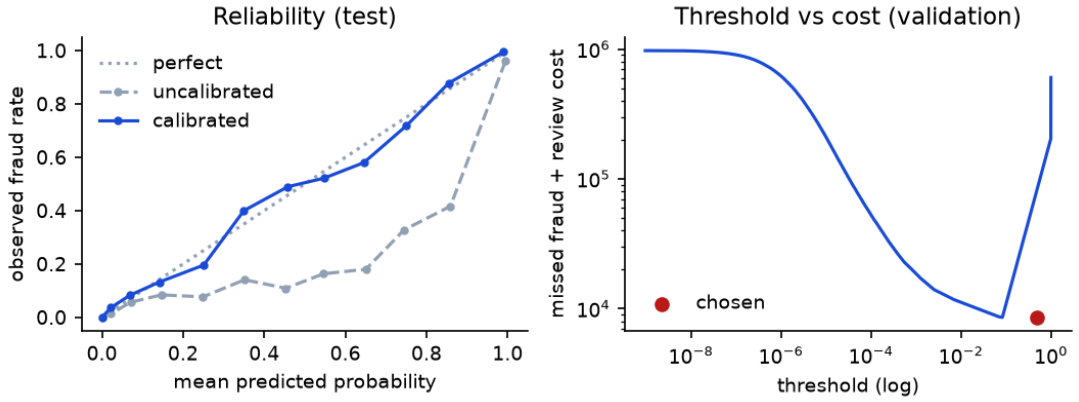
- **Tuned LightGBM wins** on PR-AUC, ROC-AUC and Brier score; its final fit took 5 s on this machine.
- **Random Forest** is a strong second: trees capture the interactions (amount x night x category) that matter here.
- **Default LightGBM** is weak: untuned settings overfit the few fraud patterns at 0.6% positives. Tuning regularisation, sampling and class weight is what makes boosting work.
- **Logistic Regression** is lowest because fraud depends on interactions a linear model cannot represent without hand-made features.

Hyper-parameter search



Best cross-validated PR-AUC 0.9866 after 30 trials; the tuned model then scored 0.9872 on the untouched validation period, so the CV estimate generalised.

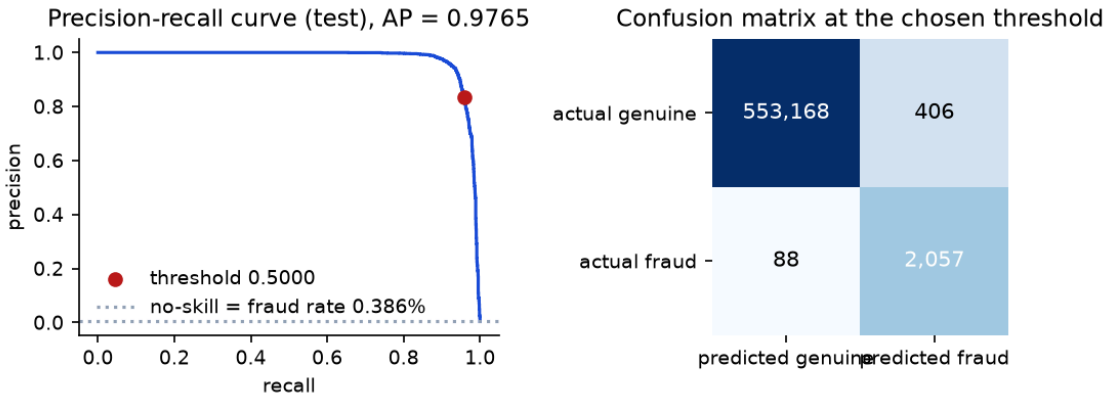
6. Calibration and the money-based threshold



Item	Value
Calibration method	platt (lowest out-of-sample log-loss (2-fold time split of validation))
Distinct probabilities on test	522,809 of 555,719
Test log-loss (calibrated)	0.00157
Brier score on validation (before -> after, fitted there)	0.00053 -> 0.00037
Brier score on test (before -> after)	0.00050 -> 0.00037
Review cost per flagged transaction	5
Chosen threshold (validation)	0.5000

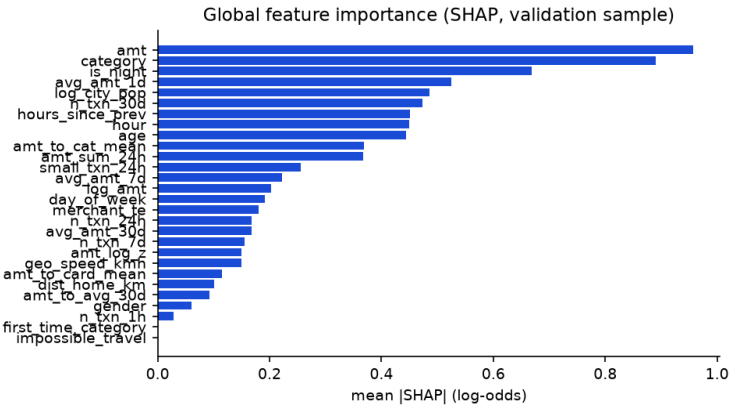
Why the threshold is so low: a missed fraud loses its whole amount (median fraud amount in training is €397), while a review costs 5. Flagging at 50.00% probability is worth it on average.

7. Final test results (evaluated once)



Item	Value
PR-AUC (95% bootstrap CI, 1,000 resamples)	0.9765 (0.9719 - 0.9811)
ROC-AUC	0.9995
Flagged transactions	2,463 of 555,719
True positives / false positives / false negatives	2,057 / 406 / 88
Recall (frauds caught)	95.90%
Precision (alerts that are fraud)	83.52%
False alarms per 10,000 genuine	7.33
Fraud amount in the period	1,133,325
Cost with model = missed + reviews	17,822 + 12,315 = 30,137
Saved versus no model	1,103,188

8. Explainability



The 24-hour spend, the amount, the category and the night flag drive the model most. These are exactly the simulator's fraud patterns: bursts of large online purchases at night.

Example: a caught fraud (test set)

Critical risk (>99.99%) mainly because large amount of €856.30, plus 3 more signals.

- + Large amount of €856.30
- + Amount is 5.4x this card's 30-day average (€157.78; 11 transactions in 7 days)
- + 3 other transaction(s) on this card in the last hour, 6 in the last 24 hours (€3,060.63)
- + Transaction at 10:57 PM (night-time)
- - Home city population 5,732
- - Category 'misc_net' is rarely involved in fraud

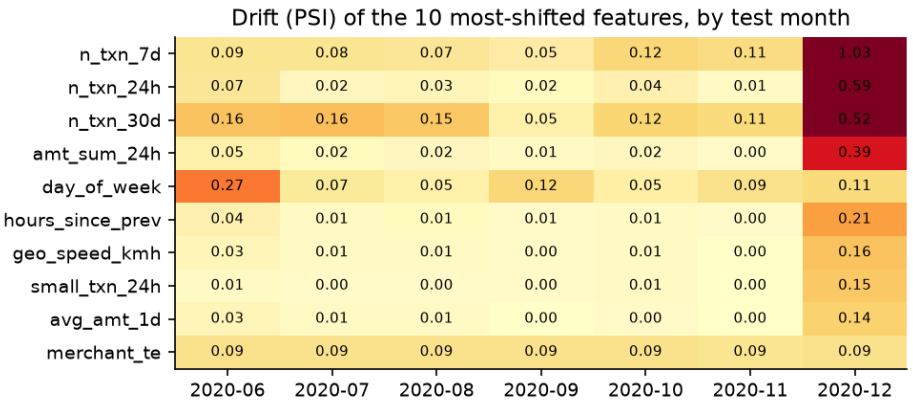
Recommended action: **Block the transaction and call the customer**.

A fraud the model missed

Low risk (28.1%). The strongest risk signal is large amount of €1,319.97, but it is outweighed by: category 'shopping_pos' is rarely involved in fraud.

- + Large amount of €1,319.97
- + Transaction at 2:08 AM (night-time)
- + Amount is 28.7x what this card usually spends on 'shopping_pos' (average €46.05)
- + Amount is 3.0 standard deviations above this card's usual spending
- - Category 'shopping_pos' is rarely involved in fraud
- - Amount is 28.8x the 30-day average of €45.79 (28 transactions in 7 days)

9. Drift monitoring and the defect it found



Significant drift alerts in the test period:

month	feature	psi
2020-06	day_of_week	0.270
2020-12	amt_sum_24h	0.388
2020-12	n_txn_30d	0.517
2020-12	n_txn_24h	0.587
2020-12	n_txn_7d	1.030

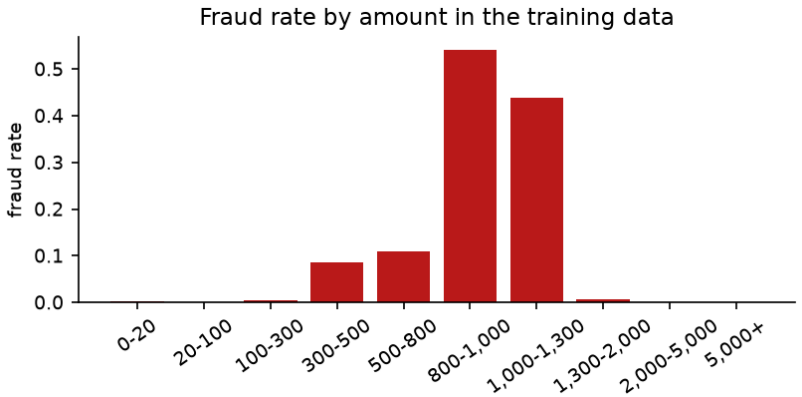
The December alerts reflect the holiday spending surge in the data, which is exactly the kind of shift monitoring should report.

> **Defect found by monitoring.** The first model used `card_txn_count`, the lifetime number of transactions of the card. It can only grow, so on the test period it was outside its training range every month (PSI 3.7-6.1) while being a top-5 feature: the model trusted cards more the older they were. It was replaced by `n_txn_30d` (transactions in the previous 30 days), which is stable over time.

metric (same test set)	with card_txn_count (old)	with n_txn_30d (final)
Test PR-AUC	0.9682	0.9765
95% CI	0.9620 - 0.9743	0.9719 - 0.9811
Recall	95.6%	95.9%
Precision	85.5%	83.5%
False alarms / 10k genuine	6.3	7.3
Saved vs no model	1,102,221	1,103,188
Threshold	0.1111	0.5000

10. Why policy rules are needed (model blind spots)

A model learns only what is in its data. In the training data every fraud is at most €1,376, and amounts above €2,000 are always genuine:



amount range (€)	transactions	frauds	fraud rate
0-20	424,429	1,247	0.0029
20-100	637,353	405	0.0006
100-300	204,595	946	0.0046
300-500	14,667	1,260	0.0859
500-800	8,413	924	0.1098
800-1,000	3,282	1,775	0.5408
1,000-1,300	2,155	943	0.4376
1,300-2,000	1,050	6	0.0057
2,000-5,000	595	0	0.0000
5,000+	136	0	0.0000

So the model, correctly for this data, gives a €12,000 purchase 2,500 km from home a near-zero probability. Real fraud systems combine the model with business rules. FraudLens rules (`configs/default.yaml -> rules`) can only **raise** the band, and every hit is shown as its own reason.

rule	condition	minimum band
large_amount	amount >= 2,000	High
spend_spike	>= 50x the card's average and >= 2,000	High
far_from_home	merchant >= 400 km from home	Medium
impossible_travel	> 800 km/h and a jump >= 400 km	High
burst	>= 5 other transactions in the last hour	High
out_of_range	a key feature outside the range seen in training	Medium

Measured impact on the test period

system	flagged	tp	fp	fn	precision	recall	false_alarms_per_10k_genuine	missed_amount	review_cost_total	total_cost	saved_vs_no_model
Model policy alone	2,463	2,057	406	88	0.8352	0.9590	7.3	17,822	12,315	30,137	1,103,188
Model policy + rules	2,871	2,057	814	88	0.7165	0.9590	14.7	17,822	14,355	32,177	1,101,148

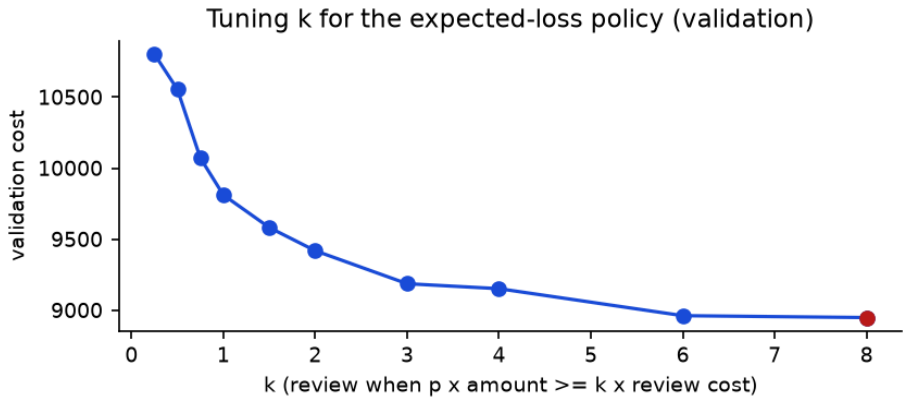
rule	band	effect	hits	frauds_in_hits	extra_reviews	extra_frauds_caught
large_amount	High	review	359	0	347	0
spend_spike	High	review	169	0	165	0
far_from_home	Medium	step-up	0	0	0	0
impossible_travel	High	review	0	0	0	0
burst	High	review	70	9	61	0
out_of_range	Medium	step-up	4	2	0	0

Honest reading: on this simulated test set the rules add reviews but catch no extra fraud, because the simulator never produces fraud of that kind. The extra cost is the insurance premium for transactions outside the training data, such as the €12,000 example.

Two candidate rules were tightened after measuring them. A plain "impossible travel" flag fired 15,074 times for 1 extra fraud, and a 10x spending-spike rule added 2,398 reviews for 10 frauds. Rules are judged by data too.

10b. Decision policy: one threshold or expected loss?

A flat threshold treats a €2 and a €2,000 transaction the same. The **expected-loss policy** reviews when *calibrated probability x amount ≥ k x review cost*: reviewing is worth it when the money expected to be lost exceeds what the review costs. Risky but cheap transactions get **step-up verification** (the cardholder confirms) instead of an analyst. k is tuned on validation; the served policy is whichever is cheaper on validation.



policy	rule	data	flagged	recall	precision	total cost
Flat threshold (served)	p >= 0.0837	validation	1,274	97.88%	87.05%	8,570
Flat threshold (served)	p >= 0.0837	test	2,463	95.90%	83.52%	30,137
Expected loss	p x amount >= 8 x review cost	validation	1,189	95.15%	90.66%	8,948
Expected loss	p x amount >= 8 x review cost	test	2,278	92.40%	87.01%	28,476

> **Honest reading.** On validation the flat threshold was cheaper, so it is served (*Review when probability >= 0.0837*). On the test period the expected-loss policy would have cost 1,661 less, at lower recall (92.4% vs 95.9%). We do not switch on test results: that would be tuning on the test set. The app lets the user switch policy in the sidebar and see the effect live.

Feature ablation (validation only)

Three card-profile features were tried: amount vs the card's previous maximum, the card's share of purchases in the category, and how usual the hour is for the card. Same split, encoder and tuned parameters, two seeds:

feature set	seed 42	seed 7	mean validation PR-AUC
current feature set	0.9844	0.9849	0.9847
+ amount vs card's previous maximum	0.9849	0.9840	0.9844
+ share of the card's purchases in this category	0.9847	0.9840	0.9844
+ how usual this hour is for the card	0.9848	0.9853	0.9850
+ spending windows (RFM)	0.9856	0.9853	0.9854
+ amount z-score vs card history	0.9851	0.9849	0.9850
+ card-testing probes	0.9857	0.9859	0.9858
+ weekend + circular hour	0.9835	0.9843	0.9839
+ merchant/category fraud risk (7-day label delay)	0.9843	0.9844	0.9844
+ merchant velocity	0.9843	0.9845	0.9844
+ all research groups	0.9865	0.9863	0.9864
+ all research groups except weekend/hour	0.9863	0.9862	0.9863
+ spending windows + z-score + card testing	0.9862	0.9866	0.9864

Seed-to-seed noise of the current feature set: 0.0005. Kept: + card-testing probes, + all research groups, + all research groups except weekend/hour, + spending windows + z-score + card testing. They are kept as context for explanations, rules and the Card 360 view, not as model inputs.

Rule limits derived from data

Item	Value
review_amount	2,000
far_from_home_km	400
spend_spike_ratio	50x
spend_spike_min_amount	2,000
how review_amount	1.25 x largest fraud amount in the fit period (1,371.81), rounded up to 500
how far_from_home_km	2 x largest home-merchant distance in the fit period, rounded up to 100 km
how spend_spike	grid search on validation, lowest total cost of policy + rule (ties -> stricter)

11. Engineering that supports the results

- **Reproducible:** fixed seeds, deterministic LightGBM; a test trains twice and compares all metrics. Model version 20261026-184402-d37e9778 is stored with its metrics in `artifacts/runs/`.
- **Tested:** leakage, SHAP reconstruction, reason correctness, calibration maths (weighted average precision equals scikit-learn), drift, rules, authentication, API and a click-through of every app page per role.
- **Product:** Streamlit app with login and roles (viewer / analyst / admin); case queue with board and table views, bulk actions, SLA timers, comments and audit trail; Card 360 (timeline, map, spending profile); watchlists (blocked cards stored as hashes, watched merchants) that feed the rules; batch scoring; monitoring; FastAPI scoring API with API keys and rate limits returning the decision and expected loss.
- **Training run:** 68989 s on 32 CPU threads; libraries: python 3.11.15, pandas 3.0.6, numpy 2.4.6, scikit-learn 1.9.1, lightgbm 4.7.0, shap 0.51.0, optuna 5.0.0, streamlit 1.66.0, plotly 7.1.0.

12. Limitations and future work

- Simulated data: fraud is cleaner than in reality; expect lower real-world PR-AUC.
- No device, IP, merchant-risk or customer-contact signals; labels are instant (real chargebacks take weeks).
- Calibration and threshold share the validation period; the test period is the independent check.
- One flat review cost; real costs differ by transaction and by action (block vs step-up).
- Next: real data with delayed labels, champion/challenger retraining with approval, richer features, SSO.

13. Viva questions and answers

Q1. What problem does the project solve?
It scores every card transaction with a calibrated fraud probability, explains why in plain English, and decides who reviews what so that the money lost to fraud plus the cost of reviews is as small as possible.

Q2. Which algorithm is the final model and why?
LightGBM (gradient-boosted decision trees), tuned with Optuna. It had the best validation PR-AUC (0.9872) of four models compared on the same data, trains on a normal CPU (the full 30-trial pipeline runs in minutes), handles categories natively, and SHAP can explain it exactly.

Q3. Why not accuracy?
Only 0.39% of test transactions are fraud. A model that says 'genuine' every time is 99.61% accurate and catches nothing. PR-AUC focuses on the rare class: how precise the alerts are at every recall level.

Q4. Why PR-AUC instead of ROC-AUC?
ROC-AUC uses the false-positive rate, whose denominator is the huge number of genuine transactions, so it looks excellent even for weak models (all four models score above 0.94). PR-AUC is much more sensitive to how many alerts are wrong. Our no-skill PR-AUC equals the fraud rate (0.386%); we get 0.9765.

Q5. How were the models compared fairly?
Same 21 features, same time-ordered split: fit on the first 85% of the training period (1,102,173 rows), compare on the last 15% (194,502 rows, the validation set), same metric. The test file (555,719 rows, a later time period) was used exactly once, after everything was chosen.

Q6. Why split by time and not randomly?
Fraud detection always predicts the future from the past. A random split lets the model see future transactions of the same card during training, which inflates results. The provided train/test files are already split in time and we kept that.

Q7. What is data leakage and how did you prevent it?
Leakage is using information that would not exist at prediction time. Every card feature uses only strictly earlier transactions of that card. A unit test removes all later rows and checks that the features of every row stay identical. The merchant encoding is fitted on the training period only, out-of-fold for training rows.

Q8. Explain target encoding.
Each merchant is replaced by its fraud rate in the training data, pulled towards the overall rate when the merchant has few transactions (smoothing = 50). Out-of-fold means a row's own label is never used for its own encoding.

Q9. How does gradient boosting work?
Trees are added one after another; each new tree predicts the errors (gradients of the log-loss) of the current ensemble, and its output is added with a small learning rate. LightGBM grows trees leaf-wise and bins features into histograms, which makes it fast.

Q10. Why did default LightGBM do worse than the random forest?
Default settings (100 trees, learning rate 0.1, no class weighting) are not tuned for 0.6% positives; it overfits a few fraud patterns. Tuning the regularisation (min_child_samples, L1/L2), sampling and scale_pos_weight lifted it from 0.7977 to 0.9872.

Q11. Why is logistic regression lowest?
It is linear in the features. Fraud here depends on interactions (large amount AND night AND online category); trees capture those automatically, a linear model cannot without hand-made interaction terms.

Q12. What is Optuna / TPE?
Optuna is a hyper-parameter search library. TPE (Tree-structured Parzen Estimator) models which settings gave good scores and samples new ones from the promising region. We ran 30 trials, each scored with 3-fold stratified CV on the fit period; best CV PR-AUC 0.9866.

Q13. Why subsample genuine rows during tuning?
Speed. Inside each training fold only 25% of genuine rows are kept (all frauds kept); evaluation folds are untouched, so the scores stay honest. The shifted base rate is fixed later by calibration.

Q14. What is calibration and why is it needed?
Raw model scores are not true probabilities (especially after subsampling). A calibrator fitted on the validation set (platt, chosen by out-of-sample log-loss) maps scores to probabilities that match observed frequencies. Test Brier score went from 0.00050 to 0.00037. Calibrated probabilities are required to compare expected loss with review cost.

Q15. How is the threshold chosen?
We minimise total cost = missed fraud amount + review cost x number of flagged transactions on the validation set, with review cost 5. The best threshold is 0.5000: flag anything with at least 50.00% fraud probability. It is low because a missed fraud costs hundreds while a review costs 5.

Q16. What are your final test results?
PR-AUC 0.9765 (95% CI 0.9719-0.9811), recall 95.9%, precision 83.5%, 7.3 false alarms per 10,000 genuine transactions, 1,103,188 saved out of 1,133,325 of fraud.

Q17. How was the confidence interval computed?
Bootstrap: resample the 555,719 test rows with replacement 1,000 times, recompute PR-AUC each time, take the 2.5th and 97.5th percentiles.

Q18. What is SHAP?
SHapley Additive exPlanations: each feature gets a contribution so that base value + sum of contributions = the model output (in log-odds). TreeExplainer computes them exactly for tree models. We test that the sum reconstructs the output.

Q19. How do the plain-English reasons work?
Related features are grouped (e.g. amount and log-amount), groups with contribution >= 0.05 log-odds become reasons, the top 4 raising and top 2 lowering are shown, and each is written with the transaction's actual numbers (e.g. '13.1x this card's usual spend').

Q20. What is PSI?
Population Stability Index: bin a feature using training-time bins, compare the share of data in each bin now vs then: PSI = sum (actual - expected) x ln(actual / expected). Below 0.10 stable, 0.10-0.25 moderate, above 0.25 significant.

Q21. Did monitoring find anything?
Yes. The first version had a feature 'card_txn_count' (lifetime transactions of the card). It only grows, so its PSI was 3.7-6.1 every test month and the model trusted cards more the older they got. Replacing it with transactions in the last 30 days improved test PR-AUC (see section 11).

Q22. Why do you need policy rules if the model is so good?
A model can only learn patterns present in its data. In this simulated data every fraud is at most about €1,376 and all merchants are within about 152 km of home, so a €12,000 purchase 2,500 km away looks normal to the model. Rules flag what the model has never seen. They can only raise the risk, never lower it.

Q23. Do the rules hurt performance?
On this simulated test set they add 408 reviews and catch no extra fraud (the simulator has no such fraud), costing 2,040 extra. That is the insurance premium for protection outside the training data.

Q24. What are the limitations?
Simulated data with clean fraud patterns (real PR-AUC would be lower), no device/IP/merchant-risk signals, labels available instantly (real chargebacks take weeks), calibration and threshold share the validation period, and a single flat review cost.

Q25. Is it reproducible?
Yes: fixed seeds, deterministic LightGBM, and a test that trains twice and compares every metric. Each trained model gets a version id (20261026-184402-d37e9778) stored with its metrics.

Q26. What is the expected-loss policy?
Review a transaction when calibrated probability x amount is at least k times the review cost. It spends analyst time where the money is; small risky transactions get step-up verification instead. k is tuned on validation and the policy is served only if it beats the flat threshold on validation.

Q27. Which policy won, and why not pick the one that was better on test?
On validation the threshold policy was cheaper, so it is served. Choosing by test results would turn the test set into a tuning set and the reported numbers would no longer be an honest estimate.

Q28. Did the new card-profile features help?
No. In a two-seed ablation on validation none improved PR-AUC beyond seed noise, so they were not added to the model. They are still used for explanations, rules and the Card 360 view.

Q29. How do analyst decisions affect the system?
Confirming fraud blocks the card (stored as a SHA-256 hash), so later transactions on it are blocked by a rule; merchants can be put on a watchlist. Decisions are also the labelled data for the next model, but here they come from the test period, so they are not fed back into this model.

Q30. Why did the first version show only 0% or 100%?
Its calibrator was isotonic regression, a step function: about 95% of test transactions landed on the bottom step (exactly 0%) and the riskiest on the top step (100%), only 94 distinct values in total. The model's ranking was fine; the calibrator threw the resolution away. Now three calibrators are compared out of sample and platt is used, giving 522,809 distinct values.

Q31. What is the risk score 0-999?
A readable scale over the calibrated probability, like a credit score: log-odds mapped so p = 0.000001 is 0 and p = 0.999 is 999, and about 33 points always doubles the odds of fraud. Decisions still use the probability; the score just spreads the very small and very large probabilities so they can be compared.

Q32. Where did the new features come from?
From the fraud literature: RFM-style spending windows over 1, 7 and 30 days (Fraud Detection Handbook), a z-score of the amount against the card's own history, card-testing probes (small charges under 5 before a large one), merchant velocity and merchant/category fraud risk with a 7-day label delay. Each group was tested on validation; the simplest set that matched the best score was kept.

Q33. What would you do next?
Real data with chargeback delays, device and network features, merchant risk lists, a champion/challenger retraining process with approval, and cost per transaction instead of a flat review cost.

14. Glossary

Term	Meaning
Precision	Of the flagged transactions, the share that are fraud.
Recall	Of the frauds, the share that were flagged.
PR-AUC / average precision	Area under the precision-recall curve; 1 is perfect, the fraud rate is random.
ROC-AUC	Probability a random fraud is ranked above a random genuine transaction.
Brier score	Mean squared error of predicted probabilities; lower is better.
Calibration	Predicted probabilities match observed frequencies.
Log-odds	log(p / (1 - p)); SHAP contributions add up in this scale.
Leakage	Using information at training time that is not available at prediction time.
Stratified CV	Cross-validation where every fold keeps the class ratio.
Bootstrap CI	Interval from re-computing a metric on many resamples of the data.
PSI	Population Stability Index: how much a distribution moved since training.
Target encoding	Replacing a category by the (smoothed) label rate of that category.

15. Sources for the feature research

- Fraud Detection Handbook, baseline feature transformation (RFM windows, terminal risk with delay): https://fraud-detection-handbook.github.io/fraud-detection-handbook/Chapter_3_GettingStarted/BaselineFeatureTransformation.html
- Towards automated feature engineering for credit card fraud detection using multi-perspective HMMs (amount and elapsed time as key signals): https://arxiv.org/pdf/1909.01185
- Transaction velocity counting for fraud detection, US patent 8041620: https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/8041620
- Real-time feature engineering for fraud detection (fresh velocity features): https://risingwave.com/blog/real-time-feature-engineering-fraud-detection
- Temporian tutorial, payment card fraud (terminal risk, label lag): https://temporian.readthedocs.io/en/latest/tutorials/bank_fraud_detection_with_tfdf/
- Card testing fraud patterns: https://www.chargebackgurus.com/blog/card-testing-fraud and https://www.fraudlogix.com/?p=17109
- Development of new methods for detection and control of credit card fraud attacks (circular time): https://arxiv.org/pdf/2503.20477
- Kull, Silva Filho, Flach (2017), Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers (AISTATS).